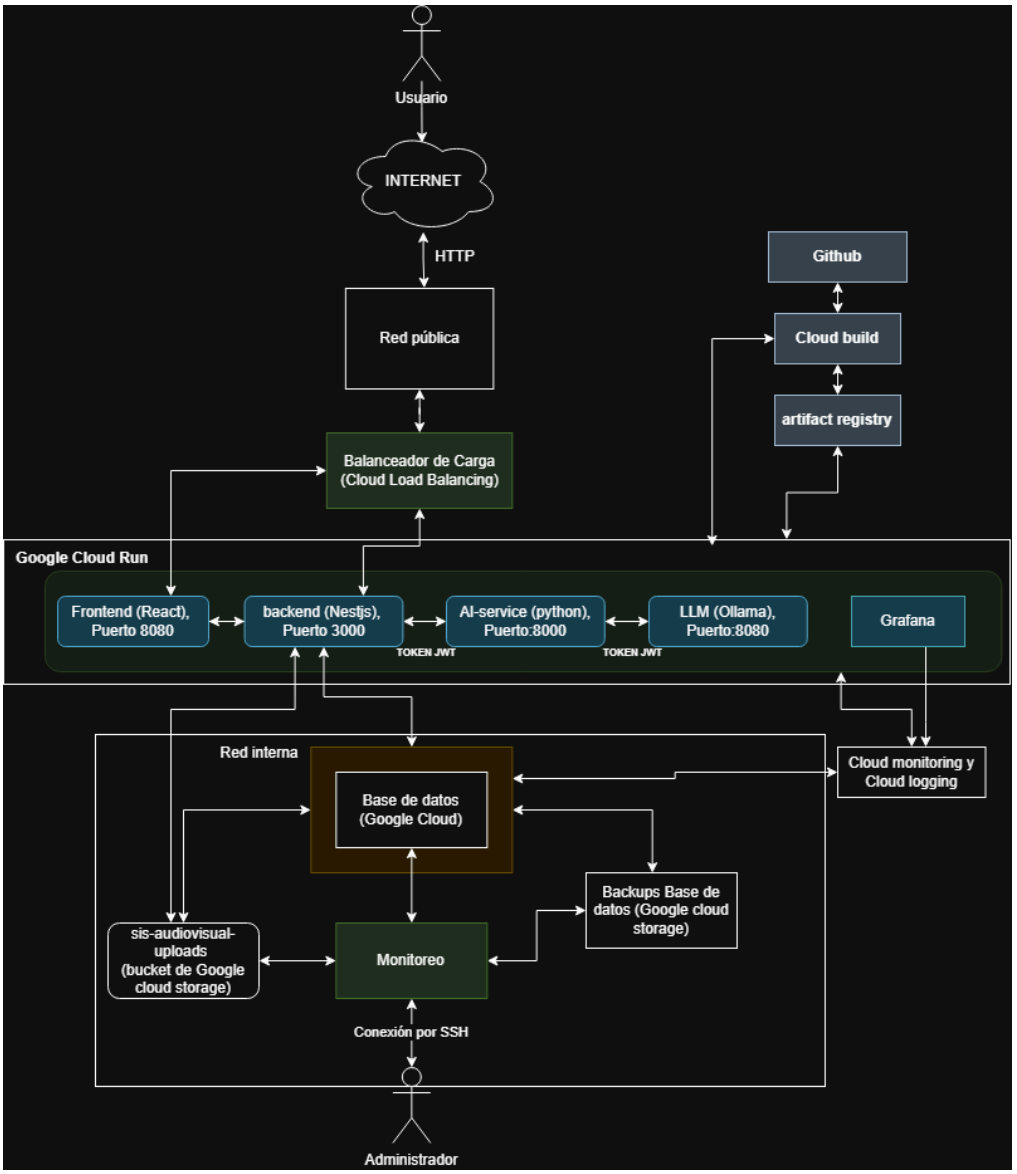
Proyecto Unidad 2 Administración de Sistemas Computacionales:
Sistema audio visual para CORES

Integrantes: Carlos Hormazabal, Vicente Farías

1. Introducción

Este documento detalla la arquitectura y las configuraciones de red implementadas para el despliegue del Sistema Audiovisual para CORES. El objetivo del proyecto es entregar una plataforma dinámica y segura, capaz de gestionar eventos en tiempo real, tales como votaciones, control de asistencia y asignación de turnos de palabra. Para garantizar alta disponibilidad y protección continua, diseñamos una infraestructura en Google Cloud fundamentada en la segmentación de redes y un sistema automatizado de respaldos.

2. Diagrama Arquitectura:



3. Evolución de la arquitectura y Distribución de las Redes:

En la primera versión del sistema, la infraestructura dependía de máquinas virtuales dentro del servicio de google cloud llamado Compute Engine, las cuales estaban segmentadas rígidamente en una red pública y una red interna privada para proteger los datos. Sin embargo, esta arquitectura generaba un alto sobrecosto operativo al mantener servidores encendidos 24/7 y presentaba desafíos de escalabilidad.

En esta segunda fase del proyecto, la arquitectura evolucionó hacia un modelo Serverless utilizando Google Cloud Run. La segmentación tradicional en una red pública y una red interna fue reemplazada parcialmente por una arquitectura de Confianza Cero para los microservicios. Bajo este enfoque, el sistema asume que ninguna conexión es segura por defecto, incluso si proviene de la misma red interna, y exige una verificación continua y estricta en cada interacción. Por ello, la seguridad de los contenedores no depende de IPs privadas, sino de dos controles. Por un lado, el Frontend y el Backend restringen su Ingress a “Interno + Load Balancing”, de modo que solo reciben tráfico a través del Balanceador de Carga, que es el único punto de entrada desde Internet. Por otro lado, IA-Service y Ollama exigen autenticación obligatoria mediante IAM, es decir, cada servicio llamador envía un token de identidad OIDC (formato JWT) emitido por Google, y solo la cuenta de servicio con el rol Cloud Run Invoker puede invocar al servicio destino. Grafana, por su parte, es accesible únicamente mediante inicio de sesión con credenciales almacenadas en Secret Manager. La red privada (VPC) se mantuvo de forma exclusiva para resguardar la Base de Datos (Cloud SQL), que solo posee IP privada y a la que el Backend accede mediante Direct VPC egress, y la máquina virtual de monitoreo. Esta última funciona como Bastion Host para realizar la conexión segura a la base de datos por SSH y, desde allí, se siguen automatizando los respaldos hacia Cloud Storage mediante cron y una cuenta de servicio, garantizando el aislamiento lógico de la capa de persistencia.

Archivos multimedia:

En el transcurso de la Unidad 1, el almacenamiento de archivos multimedia se realizaba en un directorio local llamado “uploads” dentro de cada máquina virtual, lo cual generaba graves problemas de sincronización de datos y errores 404 cuando el balanceador de carga (Nginx) administraba el tráfico entre distintas máquinas. Además, los intentos de sincronización consumían recursos innecesarios, para “engañar” al sistema y eludir el límite de un minuto de cron, se utilizaba un comando sleep que forzaba la conexión SSH cada 10 segundos hacia la otra máquina, provocando una importante sobrecarga de CPU.

Para solucionar este problema definitivamente, durante la Unidad 2 se delegó el almacenamiento de archivos a Google Cloud Storage mediante el uso de Buckets. Ahora, cuando un usuario sube o reproduce un video, el cliente interactúa de forma

directa con Cloud Storage, lo que evita que los archivos pesados transiten por la memoria del Backend. Esto libera por completo a los contenedores de Cloud Run de cargas y elimina de raíz los problemas de sincronización de la versión anterior.

Administración y Monitoreo:
La administración segura de la base de datos se realiza a través de conexiones SSH hacia la máquina virtual de monitoreo, que está en la red privada, manteniendo así el aislamiento de los datos. Además, la observación del sistema se potenció con la integración de Grafana, que se conecta nativamente a Google Cloud Monitoring y Google Cloud Logging y permite monitorear en tiempo real el consumo de CPU, la memoria, el tiempo de respuesta y los logs de los contenedores de Cloud Run, lo cual es importante para asegurar su correcto funcionamiento.

4. Decisiones de diseño.

Para este proyecto se eligió una arquitectura Serverless utilizando Google Cloud Run, en lugar de la infraestructura anterior, ya que la arquitectura serverless es superior principalmente por la reducción de costos operacionales, la escalabilidad automática y la simplificación de la administración de la infraestructura.

Para un sistema web compuesto por un frontend y un backend, Cloud Run permite desplegar ambos servicios mediante contenedores Docker, sin necesidad de administrar directamente sistemas operativos, configurar servidores web o mantener máquinas virtuales funcionando permanentemente.

5. Microservicios

Frontend: Microservicio encargado de desplegar la interfaz web de la aplicación. Se comunica únicamente con el backend, que coordina el resto de los servicios del sistema, como IA-Service y Ollama.

Backend: Microservicio que actúa como intermediario entre el frontend y los demás servicios del sistema. Se encarga de gestionar las solicitudes, coordinar las operaciones y centralizar la lógica de negocio de la aplicación.

IA-Service: Microservicio en Python (FastAPI) que transcribe el audio de los videos de las sesiones mediante la librería faster-whisper y que recibe las consultas del chat, validando que se relacionen con la sesión antes de enviarlas a Ollama. Solo el Backend está autorizado para invocarlo.

Ollama: Microservicio encargado de alojar y ejecutar el modelo de lenguaje llama3.2:3b, que responde preguntas sobre las transcripciones de las sesiones. Solo IA-Service está autorizado para invocarlo.

Load-balancer: Componente encargado de distribuir el tráfico entrante entre las instancias disponibles de los servicios, permitiendo mejorar la distribución de la carga y la disponibilidad del sistema.

Grafana: Microservicio de monitoreo que se conecta a Cloud Monitoring y Cloud Logging para visualizar el estado, las métricas y los logs de todos los servicios.

google-cloud-sql: Servicio de base de datos administrado en la nube, encargado de almacenar y gestionar la información del sistema, proporcionando mecanismos de respaldo, mantenimiento y alta disponibilidad.

Docker

- Dockerfile: Cada servicio de la aplicación, a excepción del Load Balancer y de Cloud SQL, dispone de un Dockerfile. Esto permite concentrar la imagen de cada servicio en un solo lugar y aporta a la modularidad de la aplicación.

ai-service/Dockerfile: Crea un entorno ligero de Python para el servicio de Inteligencia Artificial. Se encarga de instalar herramientas clave como ffmpeg para procesar videos, configura rutas para guardar modelos en memoria caché y crea un usuario especial sin permisos de administrador por motivos de seguridad.

backend/Dockerfile: Utiliza un proceso “Multi-stage” para que el servidor Node.js final sea lo más rápido y liviano posible. Primero instala las librerías, luego compila el código y prepara la conexión a la base de datos, finalmente mueve solo el código ya procesado a un contenedor limpio que se ejecuta bajo un usuario seguro.

frontend/Dockerfile: Al igual que el backend, usa múltiples etapas para optimizar la interfaz de React. En la primera etapa toma el código fuente y lo empaqueta en archivos estáticos; en la segunda, coloca estos archivos dentro de un servidor web Nginx en su versión sin privilegios de administrador, que se encarga de servir la página a los usuarios.

ollama/Dockerfile: Prepara el motor para ejecutar modelos de lenguaje. Durante la construcción de la imagen levanta un servidor temporal para descargar el modelo llama3.2:3b, de modo que al iniciar el servicio el modelo ya está instalado y responde sin descargas adicionales.

grafana/Dockerfile: Configura una plataforma de monitoreo visual partiendo de la imagen oficial de Grafana 11.6.0. Este contenedor automatiza por completo su despliegue al incorporar de antemano las configuraciones de datasources y los dashboards, además de instalar el plugin de Google Cloud Logging. Gracias a esto, Grafana arranca totalmente conectado a Cloud Monitoring y Cloud Logging, eliminando la necesidad de configuración manual. Para reforzar la seguridad y la estabilidad, el contenedor incluye una directiva HEALTHCHECK, deshabilita el acceso a usuarios anónimos y se ejecuta bajo un usuario restringido (UID 472) sin

privilegios de administrador, garantizando un entorno de observabilidad seguro y controlado.

6. Aplicación de la ia y transcripción.

Módulo de inteligencia artificial: Para el módulo de inteligencia artificial se utilizó “Ollama”, Ollama es una herramienta que permite ejecutar y administrar modelos de lenguaje grandes de manera local o en infraestructura propia, proporcionando una API. En esta aplicación se utilizó para obtener el contexto de las sesiones con el objetivo de aclarar dudas respecto al acta a través de un Chat.

Transcripción: Para la transcripción de las sesiones se utilizó la librería faster-whisper, que convierte el audio de los videos en texto.

7. Monitoreo.

Grafana es una herramienta de monitoreo y visualización de datos que permite supervisar el estado y rendimiento de los distintos servicios del sistema mediante dashboards. Facilita el seguimiento de métricas, la identificación de posibles problemas y el análisis del comportamiento de la infraestructura en tiempo real, permitiendo detectar anomalías y mejorar la gestión del sistema.

8. Orquestación.

En producción, la orquestación de los contenedores la realiza Google Cloud Run, que se encarga de desplegar cada servicio, escalarlo automáticamente según la demanda y reemplazar las instancias que fallan, por lo que no se requiere un orquestador adicional. Cada servicio se construye a partir de su propio Dockerfile y se despliega de forma independiente mediante Cloud Build y el Artifact Registry.

9. Análisis económico

1. Costos de Desarrollo (Inversión Inicial) La categoría tiene en cuenta el esfuerzo humano y técnico requerido para diseñar, construir y desplegar la solución desde cero hasta su puesta en marcha.

- **Ingeniería y Arquitectura de Software:** Contempla el trabajo de 2 Ingenieros de Software durante 6 semanas, aproximadamente 120 horas cada uno. Esto abarca el diseño de la arquitectura, la creación de los Dockerfiles, la orquestación, las pruebas de integración y el despliegue en Google Cloud. El valor estimado es de $ 2.500.000 CLP.
- **Gestión de Proyecto y Documentación:** Incluye análisis de los requerimientos de la licitación de Mercado Público, la redacción de los manuales técnicos, diagramas de arquitectura y documentación de los procedimientos de backup y restore. El valor estimado es de $ 150.000 CLP.
- **Total Desarrollo: $ 2.650.000 CLP (Pago único).**

2. Costos de Infraestructura Costos recurrentes por el uso de los servicios en la nube de Google, sin contar la inteligencia artificial.

- **Servicios Web, Cloud Run:** Ejecución de los contenedores del frontend, backend y grafana. Como el modelo del proyecto es serverless se cobra por petición y autoescala a cero cuando no hay tráfico, el costo es bajo. Se estiman aproximadamente unos $ 10.000 CLP / mes.
- **Persistencia de Datos, Cloud SQL:** Instancia de base de datos administrada PostgreSQL alojada de forma segura dentro de la red privada (VPC). Se estiman $ 20.000 CLP / mes.
- **Máquina Virtual y Almacenamiento:** Uso de Compute Engine para la VM tradicional de monitoreo y almacenamiento en Cloud Storage para guardar todo lo relacionado con las sesiones, videos, transcripciones, actas, etc. Se estiman $ 12.000 CLP / mes.
- **Total Infraestructura: $ 42.000 CLP / mes.**

3. Costos Asociados al Componente de IA (Operación Mensual) Aquí se separan estratégicamente los recursos de cómputo intensivo requeridos para que los modelos de lenguaje y transcripción funcionen correctamente en la nube.

- **Motor LLM (Ollama):** Este es el contenedor más pesado de la arquitectura. Para que el modelo llama3.2:3b responda sin colapsar por falta de memoria, requiere una configuración robusta en Cloud Run con 16 GB de RAM y 8 vCPUs. Se estiman $ 25.000 CLP / mes.
- **Motor de Transcripción (IA-Service):** Contenedor basado en FastAPI que ejecuta faster-whisper para procesar el audio de los videos. Consume recursos moderados de CPU durante el procesamiento. Se estiman $ 15.000 CLP / mes.
- **Total Inteligencia Artificial:** $ 40.000 CLP / mes.

4. Costos de Operación y Mantención (Mensual) Categoría que cubre los servicios necesarios para garantizar la continuidad operacional, seguridad y resiliencia del sistema a lo largo del tiempo.

- **Soporte y Monitoreo Activo:** Revisión periódica de las métricas de latencia, CPU y errores en los dashboards de Grafana, además de la gestión de alertas y resolución de incidentes técnicos. Se estiman $ 75.000 CLP / mes.
- **Mantenimiento de Software y Seguridad:** Aplicación de parches de seguridad, actualización de imágenes base en los Dockerfiles para evitar vulnerabilidades, y la gestión/rotación de políticas IAM y tokens JWT. Se estiman $ 80.000 CLP / mes.
- **Gestión de Respaldos (Backup):** Ejecución periódica de las copias de seguridad de la base de datos y simulacros de restauración para asegurar que el sistema cumple con las políticas de recuperación ante desastres. Se estiman $ 25.000 CLP / mes.

- **Total Mantención:** $ 180.000 CLP / mes.

Resumen Económico General:

- **Costo de Implementación (Único):** $ 2.650.000 CLP
- **Costo de Operación y Mantención (Mensual):** $ 262.000 CLP

10. Conclusión

La evolución de esta arquitectura hacia un modelo Serverless en Google Cloud transformó el Sistema Audiovisual para CORES, logrando un buen equilibrio entre la innovación tecnológica con Inteligencia Artificial, el control del presupuesto operativo y una infraestructura segura.

La adopción de Google Cloud Run para la orquestación de contenedores permitió implementar un sistema que escala automáticamente según la demanda, sin mantener servidores encendidos de forma permanente. Asimismo, la migración de los archivos multimedia hacia Cloud Storage resolvió los problemas de inconsistencia de datos de la versión anterior.

Finalmente, la seguridad del sistema se vio fortalecida al transicionar hacia un enfoque Zero Trust, donde las comunicaciones entre microservicios están protegidas por tokens de identidad OIDC con IAM, y el tráfico externo ingresa únicamente a través del Load Balancer. En conjunto, estas decisiones de diseño entregan una plataforma moderna, resiliente y escalable.